KLAi

# Can I trust this model? Overfitting, cross-validation and leakage

*How do we know a model that predicts crusher energy demand will still work on a crusher it has never seen?*

CML3113 | Mining and Petroleum Engineering | Week 10, class 3 of 4 | Friday 20 November 2026, 08:00 to 11:00
KLAi | International University of East Africa | Jotham Martin Wambi

# Today in one view: what you will leave with

## By the end of today you can



Explain underfitting, a good fit and overfitting (the bias-variance trade-off), and choose max_depth with a validation curve



Work out training error, validation error, the generalisation gap, and a cross-validation mean and standard deviation, by hand



Score a model with K-fold and GroupKFold cross-validation, and report the mean with its standard deviation



Name three kinds of data leakage and use a checklist that blocks them

# Your three hours: ideas first, then the lab

| | | |
|---|---|---|
| 0:00 | 10 | Recall from Class 2, and why trust matters |
| 0:10 | 40 | Concepts, part 1: overfitting, validation curve, cross-validation |
| 0:50 | 30 | Concepts, part 2: data leakage, random seed, pipeline |
| 1:20 | 15 | Break |
| 1:35 | 75 | Lab: validation curve, cross-validation, leakage, seeds |
| 2:50 | 10 | Close and how this class counts |

Concepts first, with no laptops. Then the lab, with the notebook open.

# Three questions from last time

1. Why is a majority-class baseline (a model that always predicts no) a poor model, even when its accuracy is about 80 percent?

2. In a confusion matrix, what is a false negative (FN) and what is a false positive (FP)?

3. What happened to recall and false positives when we lowered the decision threshold from 0.5 to 0.3?




Answer aloud first. Then we check together.

# Answers to the three questions from Class 2

| Question | Short answer |
| --- | --- |
| **Why is a majority-class baseline (a model that always predicts no) a poor model, even when its accuracy is about 80 percent?** | It finds none of the real positive cases, so its recall is 0. Accuracy hides that. |
| **In a confusion matrix, what is a false negative (FN) and what is a false positive (FP)?** | False negative: the model said no (negative) and the truth was yes (positive). False positive: the model said yes and the truth was no. |
| **What happened to recall and false positives when we lowered the decision threshold from 0.5 to 0.3?** | Recall rose from 1.00 to 1.00. False positives went from 1 to 2. |

# Words we use today

| | Word | What it means |
|---|---|---|
|  | **Overfitting** | The model fits the noise in the training set (it nearly memorises it) and does worse on new rows. Underfitting is the opposite: too simple to learn the pattern. |
|  | **Cross-validation** | Scoring a model K times. The rows are cut into K parts called folds. Each fold takes one turn as the validation fold (held-out fold). |
|  | **Data leakage** | Information from outside the training set reaches the model while it is trained, so the score looks better than it really is. |
|  | **Bias-variance trade-off** | Too simple a model misses the pattern (high bias, underfitting). Too flexible a model chases noise (high variance, overfitting). |
|  | **Random seed** | A fixed number (random_state in scikit-learn) that makes a random choice repeat exactly. |

# A model that looks good on paper can fail on a new machine or site



A model is built from data on four crushers and then used on a fifth crusher.

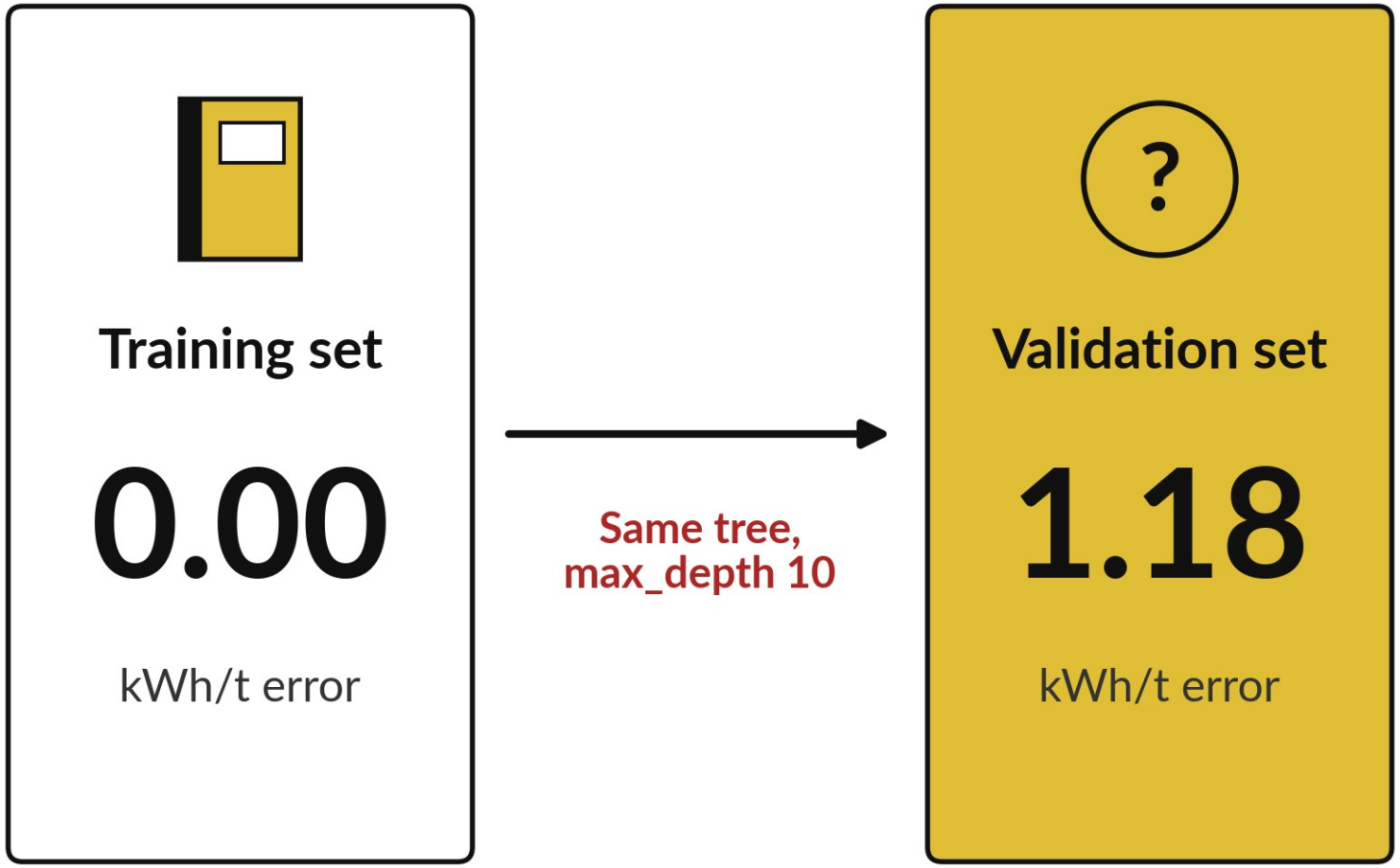


The power plan for the new pit is chosen from the model's prediction.



If the prediction is wrong, the energy bill holds a surprise.

# A model must do well on rows it has never seen

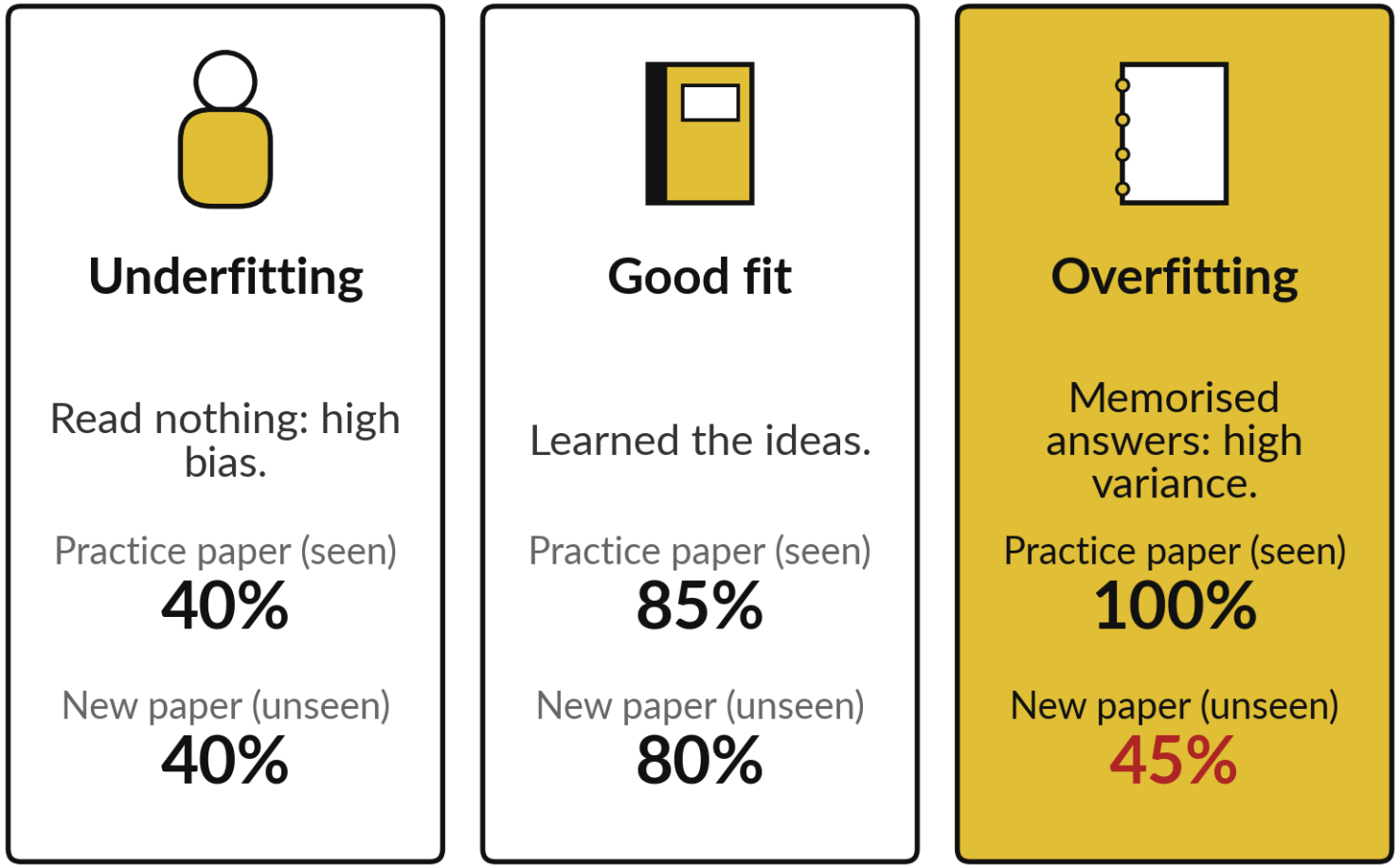



**Generalisation means doing well on held-out data: rows the model has never seen.**

The tree was trained on the training set, so its error there is almost zero.

The validation set was held out. On it the prediction error is far larger. The difference is the generalisation gap.

# Underfitting, a good fit and overfitting look like three students




**The marks in the picture are illustrative (made up). Each student stands for one kind of model.**

Practice paper = training set. New paper = held-out rows.

Bias-variance trade-off: too simple a model misses the pattern, too flexible a model chases noise.

# Reminder from Class 1: MAE and RMSE measure the prediction error

MAE: mean absolute error

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

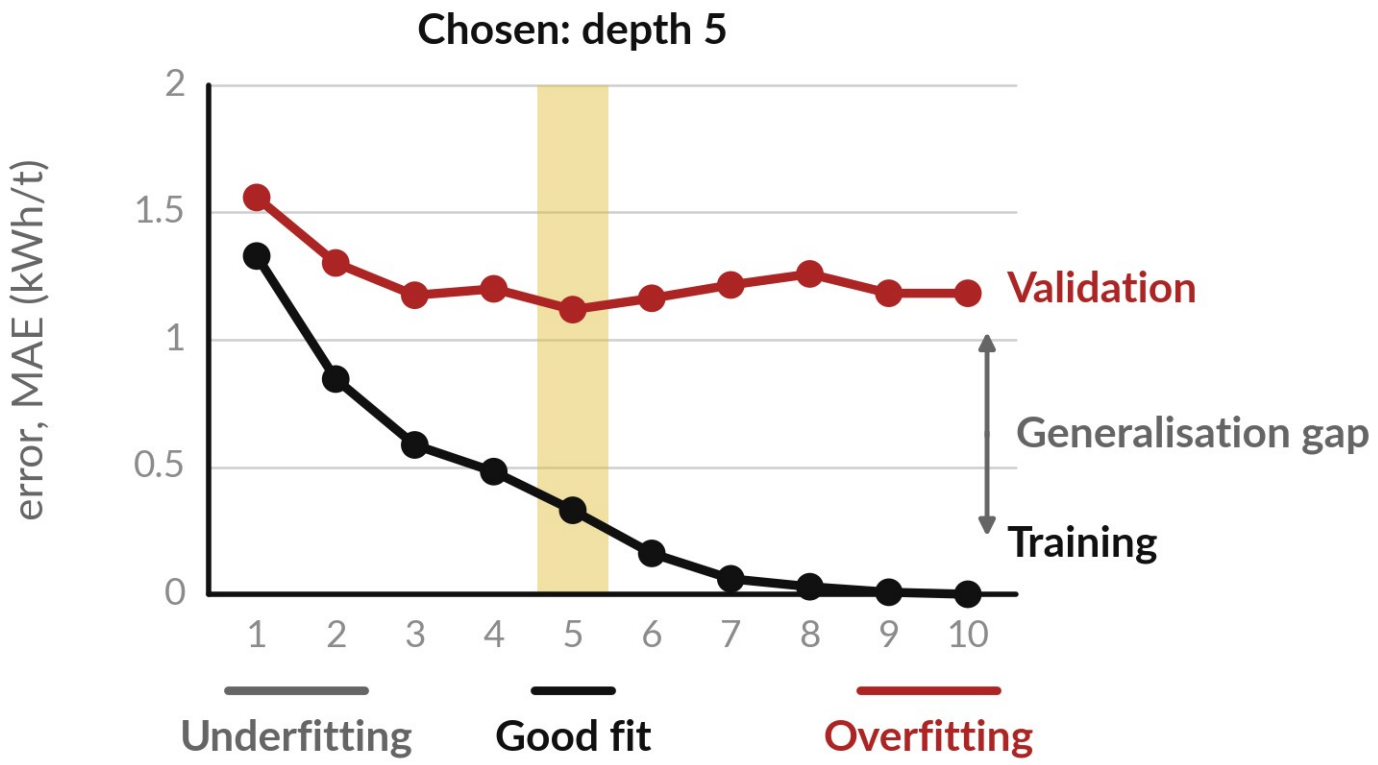
MSE and RMSE

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \quad \text{RMSE} = \sqrt{\text{MSE}}$$

| Symbol | What it means | In our data |
|---|---|---|
| **MAE** | The mean size of the error. The bars drop any minus sign | Class 1 model: 0.67 kWh/t |
| **MSE, RMSE** | MSE: mean of the squared errors. RMSE: its square root, in the target unit | Class 1 model: RMSE 0.81 kWh/t |
| **n, y, ŷ** | n rows scored; y is the actual target of row i; ŷ (y-hat) is the prediction | 15 validation rows; crusher energy demand in kWh/t |

Illustrative example (made up): errors of 1, 2 and 3 give MAE = 2, MSE = 4.67 and RMSE = 2.16.

# The validation curve: as max_depth grows, training error keeps falling but validation error stops




**A validation curve plots both errors against model complexity, here max_depth (most questions a row is asked).**

Each dot is one tree. Black: training error, falling to 0.00. Red: validation error. Between them: the generalisation gap.

The gold band is the max_depth we choose.

# Training error, validation error and the generalisation gap are short formulas

Training error

$$\mathrm{MAE}_{\mathrm{train}} = \frac{1}{n_{tr}} \sum_{i \in \mathrm{train}} |y_i - \hat{y}_i|$$

Validation error

$$\mathrm{MAE}_{\mathrm{val}} = \frac{1}{n_{va}} \sum_{i \in \mathrm{val}} |y_i - \hat{y}_i|$$

| Symbol | What it means | In our data |
|---|---|---|
| **MAE** | Mean absolute error: the average size of the prediction error. The small label train or val says which rows | Depth 10: 0.00 and 1.18 kWh/t |
| **n** | How many rows. The small label tr means training and va means validation | 58 and 15 rows |
| **i** | A row number. The sum adds up the rows of that group | Row 1, row 2, and so on |
| **y, ŷ** | The real target of row i, and the model's prediction for it. The bars \| \| drop any minus sign | crusher energy demand and the tree's answer, in kWh/t |

# The generalisation gap is the validation error minus the training error

Generalisation gap

$$\text{gap} = \text{MAE}_{\text{val}} - \text{MAE}_{\text{train}}$$

Depth 10

$$\text{gap} = 1.18 - 0.00 = 1.18 \text{ kWh/t}$$

Depth 5

$$\text{gap} = 1.12 - 0.33 = 0.79 \text{ kWh/t}$$

| Symbol | What it means | In our data |
|---|---|---|
| **gap** | Generalisation gap: how much worse the model is on new rows | Depth 10: 1.18 kWh/t |
| **MAE_val** | Validation error, from the formula before | 1.18 kWh/t at depth 10 |
| **MAE_train** | Training error, from the formula before | 0.00 kWh/t at depth 10 |

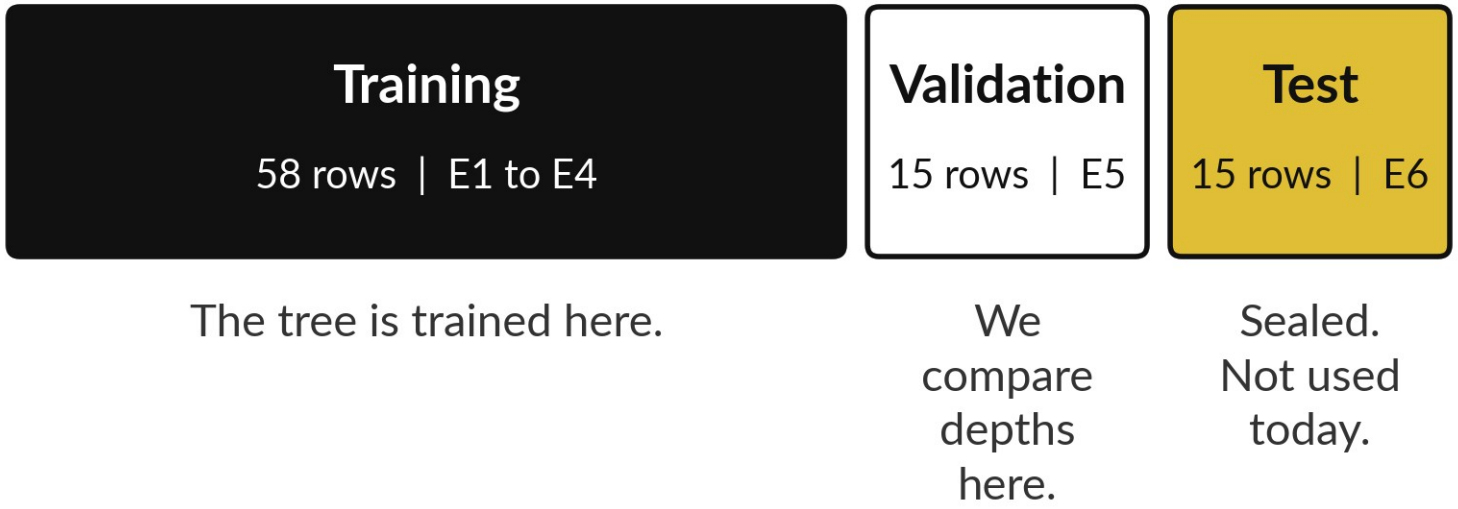
A gap near 0 means about the same error on new rows as on the training set. A big gap means overfitting.

# As max_depth grows, the training error keeps falling but the validation error does not

| max_depth | Leaf nodes | Training MAE | Validation MAE | Generalisation gap (validation minus training) |
|---|---|---|---|---|
| **1** | 2 | 1.33 | 1.56 | 0.23 |
| **2** | 4 | 0.85 | 1.30 | 0.46 |
| **4** | 14 | 0.48 | 1.20 | 0.72 |
| **5 (chosen)** | 25 | 0.33 | 1.12 | 0.79 |
| **10** | 58 | 0.00 | 1.18 | 1.18 |

MAE in kWh/t. A leaf node is a final node at the bottom of the tree, and it holds the prediction. A deeper tree has more leaf nodes. The linear regression model from Class 1 scores 0.67 kWh/t, better than every tree.

# We choose max_depth on the validation set and keep the test set sealed




**Each set is made of whole equipment groups. The picture shows how many rows each set has.**

There are only 15 validation rows, so a score can move by luck.

We never tune anything on the test set.

# Choose the simplest model within a tolerance of the best score

Tolerance limit

$$\text{limit} = 1.05 \times V_{\text{min}}$$

Chosen depth

$$d^* = \text{smallest } d \text{ with } V(d) \leq \text{limit}$$

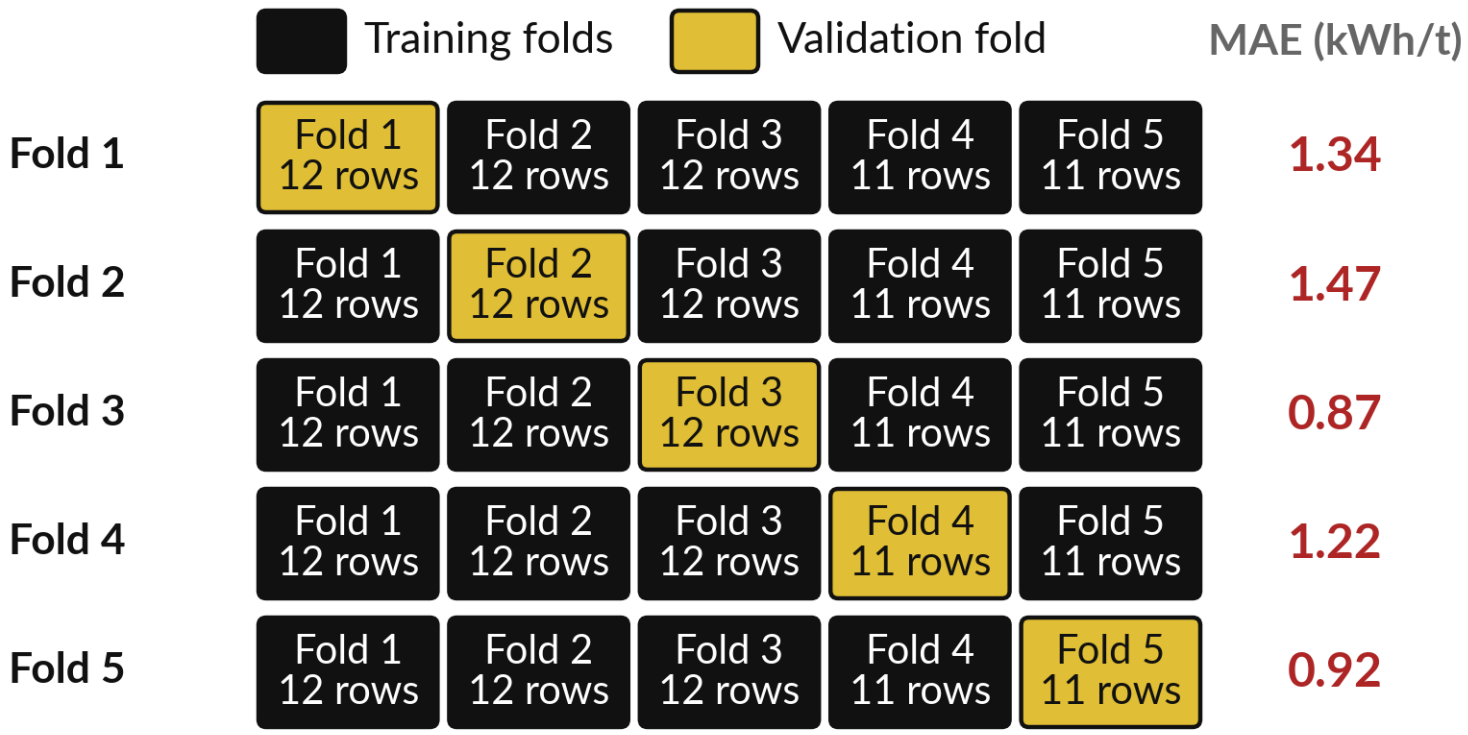
Our data

$$1.05 \times 1.12 = 1.17 \text{ kWh/t}, \; d^* = 5$$

| Symbol | What it means | In our data |
|---|---|---|
| **V(d)** | Validation MAE of the tree with max_depth d | 1.56 at depth 1, 1.18 at depth 10 |
| **V_min** | The lowest validation MAE over depths 1 to 10 | 1.12 kWh/t, at depth 5 |
| **limit** | The tolerance limit: the lowest value plus 5 percent | 1.17 kWh/t |
| **d*** | The chosen depth: the smallest d that passes | Depths that pass: 5, 6 |

A rule of thumb (parsimony: prefer the simpler model), like the one-standard-error rule. d* = chosen max_depth.

# One split can be lucky, so K-fold cross-validation lets every fold take a turn as the validation fold




**Mean 1.16 kWh/t  Standard deviation 0.24 kWh/t**

**K-fold cross-validation cuts the 58 training rows into K = 5 folds (parts of the rows), in the order the rows appear (no shuffle).**

In each row, the tree is trained on 4 folds and scored on the gold fold: the validation fold (held-out fold).

After 5 rows each fold has been the validation fold once. The 5 scores of a max_depth 4 tree are on the right.

# K-fold gives k scores, and we report their mean and their standard deviation

Mean of the k scores

$$\bar{s} = \frac{1}{k} \sum_{j=1}^{k} s_j$$

Standard deviation of the k scores

$$\sigma_s = \sqrt{\frac{1}{k} \sum_{j=1}^{k} (s_j - \bar{s})^2}$$

| Symbol | What it means | In our data |
|---|---|---|
| **s** | The MAE when fold j is the validation fold (j is the fold number) | 1.34, 1.47, 0.87, 1.22, 0.92 kWh/t |
| **k** | The number of folds, so the number of scores | 5 folds |
| **s̄** | The mean (average) of the k scores (s with a bar) | 1.16 kWh/t |
| **σ** | The standard deviation (sigma): the typical distance of a score from the mean. We divide by k, as np.std does | 0.24 kWh/t |

# By hand with four illustrative scores: the mean is 3 and the standard deviation is 0.71

| Fold | Score (MAE) | Distance from the mean | Squared distance |
|---|---|---|---|
| **1** | 2 | -1 | 1 |
| **2** | 3 | 0 | 0 |
| **3** | 3 | 0 | 0 |
| **4** | 4 | +1 | 1 |

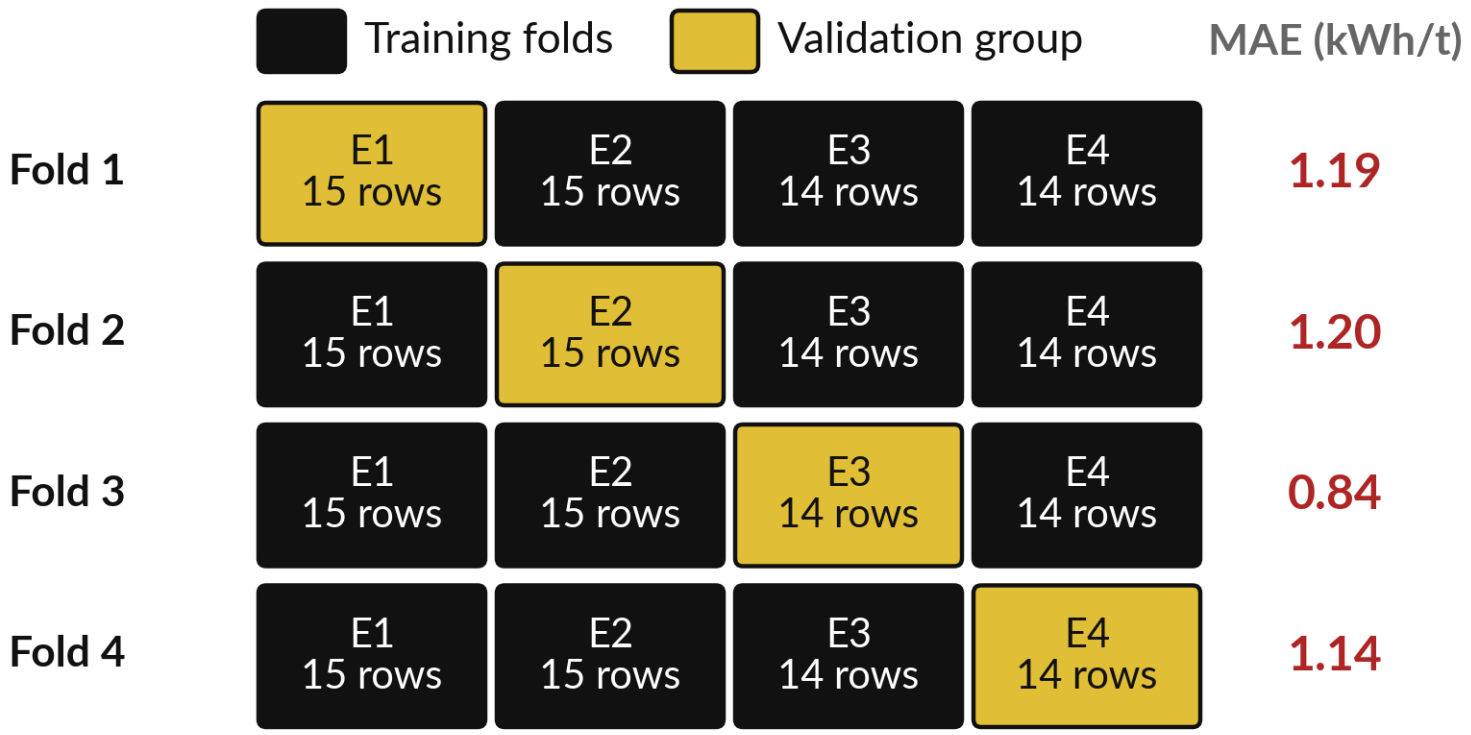
Mean = (2 + 3 + 3 + 4) / 4 = 3. Mean of the squared distances = (1 + 0 + 0 + 1) / 4 = 0.5 (the variance). The square root of 0.5 is 0.71, the standard deviation. These four scores are illustrative (made up).

# The same steps on our five real scores: mean 1.16, standard deviation 0.24 kWh/t

| Fold | Score (MAE, kWh/t) | Distance from the mean | Squared distance |
|---|---|---|---|
| **1** | 1.34 | +0.17 | 0.030 |
| **2** | 1.47 | +0.31 | 0.096 |
| **3** | 0.87 | -0.29 | 0.087 |
| **4** | 1.22 | +0.06 | 0.003 |
| **5** | 0.92 | -0.24 | 0.060 |
| **Sum** | 5.82 | 0.00 | 0.276 |

Mean = 5.82 / 5 = 1.16. Mean of squares = 0.276 / 5 = 0.055. Square root = 0.24, the standard deviation. Worked from unrounded scores, so the last digit can differ.

# GroupKFold holds out a whole equipment group in each fold, which is how the model is really used




**Mean 1.09 kWh/t  Standard deviation 0.15 kWh/t**

**GroupKFold keeps each equipment group whole. In each fold one group, E1, E2, E3 or E4, is the validation fold.**

The tree is trained on the other three groups and scored on the held-out crusher group, like a new site.

Each fold gives one score. The mean and standard deviation are worked out as before.

# GroupKFold uses the same formulas, with equipment groups as the folds

Score of group g

$$s_g = \frac{1}{n_g} \sum_{i \in g} |y_i - \hat{y}_i|$$

Mean and standard deviation over the 4 groups

$$\bar{s} = \frac{1}{4} \sum_{g} s_g, \quad \sigma_s = \sqrt{\frac{1}{4} \sum_{g} (s_g - \bar{s})^2}$$

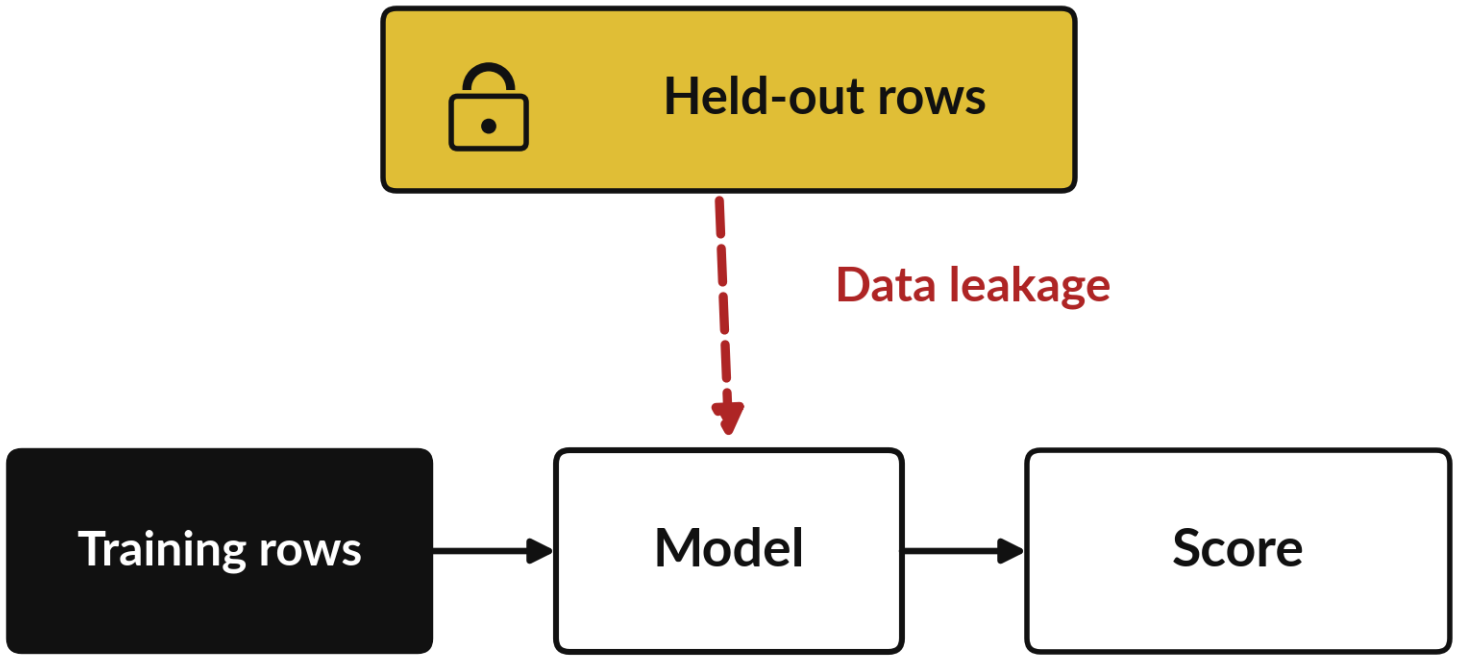
| Symbol | What it means | In our data |
|---|---|---|
| **g** | The group held out in one fold: E1, E2, E3 or E4 | E1, then E2, E3, E4 in turn |
| **n** | Number of rows in group g (the small label g) | E1 15, E2 15, E3 14, E4 14 rows |
| **s** | MAE on group g (the small label g), using a tree that was never trained on g | E1 1.19, E2 1.20, E3 0.84, E4 1.14 kWh/t |
| **$\bar{s}$, σ** | Mean of the 4 scores, and their standard deviation | 1.09 and 0.15 kWh/t |

# The same max_depth 4 tree gets three different scores, depending on how we test it

| | Way of testing | Scores (MAE) | Mean | Standard deviation |
|---|---|---|---|---|
|  | **Single validation split (group E5)** | 1.20 | 1.20 | one score only |
|  | **K-fold, k = 5 (rows in order, no shuffle)** | 0.87 to 1.47 | 1.16 | 0.24 |
|  | **GroupKFold, k = 4** | 0.84 to 1.20 | 1.09 | 0.15 |

MAE in kWh/t. Group E5 turned out harder than the average group, so its single score is higher than both averages. The standard deviation shows how far to trust each mean.

# Data leakage is information from outside the training set reaching the model




Normal path: training rows train the model, then it gets a score

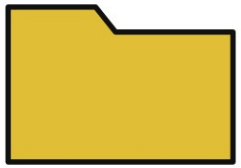
**Held-out rows must never reach the model**

**Data leakage: information from held-out data, which the model should not see, reaches the model while it is trained.**

Held-out rows are the validation or test rows. The dashed red arrow is the leakage.

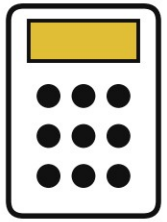
The score looks excellent, then the model fails on real new rows.

# Data leakage comes in three common kinds



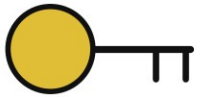
### 1 Train-test contamination

Validation rows are in the training set



### 2 Preprocessing leakage

Imputer (fills missing values) or scaler (rescales columns) fitted on all rows

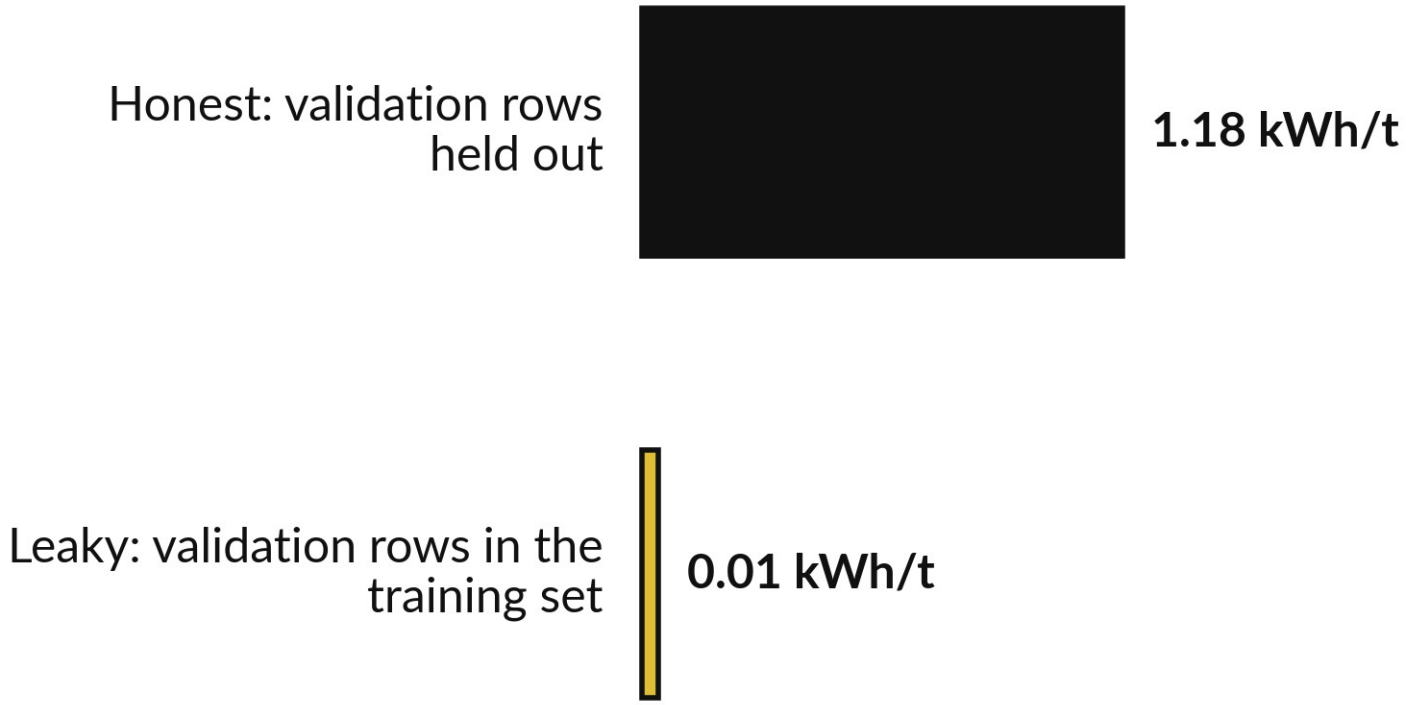


### 3 Target leakage

A feature is built from the target

All three make the score look too good.

# Train-test contamination: validation rows in the training set make the score look perfect

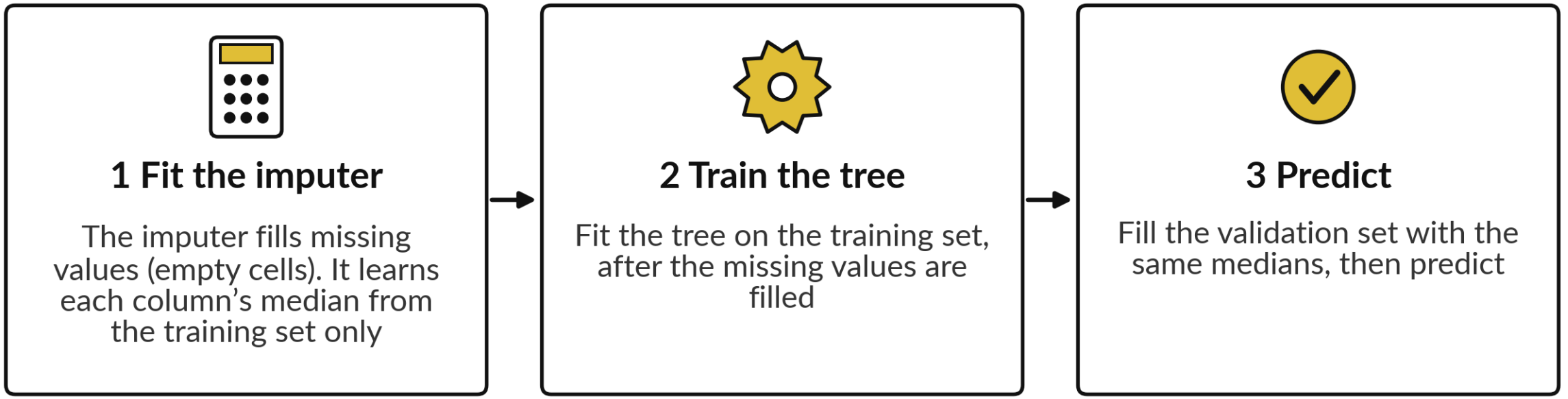



**Same max_depth 10 tree. Only the training set changed.**

In the leaky run the tree was already trained on the validation rows, so it reproduces their answers (overfitting).

The honest score is the real one.

# A Pipeline fits the imputer on the training set only, which prevents preprocessing leakage




Median of ambient_c: 26.07 from the training set, 25.37 from all rows (preprocessing leakage). Only one value is missing, so the score did not change.

# The pipeline rule: fit on the training set only, then apply the same numbers

Imputer: the median fill

$$m_j = \text{median}\{\, x_{ij} : i \in \text{train} \,\}$$

StandardScaler: mean, then z-score

$$\mu_j = \frac{1}{n_{tr}} \sum_{i \in \text{train}} x_{ij}, \qquad z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

| Symbol | What it means | In our data |
|---|---|---|
| **i, j, x, n** | i is the row, j the column, x the value in row i and column j, n the number of training rows. In preprocessing leakage, i includes validation rows | j = 1 load_pct, j = 2 ambient_c; n = 58 |
| **m** | Median of column j over the training rows | ambient_c: 26.07. From all rows: 25.37, leakage |
| **μ, σ** | Mean and standard deviation of column j over the training rows | load_pct: 59.89 and 22.73 |
| **z** | The z-score (scaled value) of row i in column j | Row 1: (59.15 - 59.89) / 22.73 = -0.03 |

# Target leakage: a feature built from the target gives a great score that cannot be repeated

| | A | B | C | D |
|---|---|---|---|---|
| | load_pct | ambient_c | target_rounded | target |
| 1 | 59.1 | 31.7 | 14 | 14.0 |
| 2 | 77.5 | 30.0 | 15 | 15.1 |
| 3 | 69.6 | 19.9 | 13 | 13.0 |

**With that feature a max_depth 4 tree scores 0.31 kWh/t. With honest features it scores 1.20 kWh/t.**

target_rounded is the target rounded to a whole number. At prediction time the target is unknown.

A feature recorded after the event is also target leakage.

# Keep this checklist: five questions before you trust a score



Was the score measured on held-out data (rows the model never saw)?



Were the imputer and the scaler fitted on the training set only (no preprocessing leakage)?



Does any feature come from the target, or only exist after the event (target leakage)?



Is the test set (group E6) still sealed?



Is the score a cross-validation mean, shown with its standard deviation?

# The same code can give different numbers, so we fix the random seed (random_state) and write it down

| random_state (the random seed) | max_depth 10 tree: validation MAE | Shuffled 5-fold: mean MAE |
|---|---|---|
| **1** | about 1.2 | about 1.1 |
| **17** | about 1.2 | about 1.1 |
| **99** | about 1.3 | about 1.0 |

MAE in kWh/t. The data and the code are the same in every row. Only the seed changed. A random seed makes a random choice repeat exactly. Your last digit can differ.

# Write down what you ran, so anyone can repeat it

**Run record**

Data file
*engineering_data.csv, 91 rows in the file*

Split
*Training E1 to E4, validation E5, test E6 sealed*

Random seed
*random_state = 17 (Step 5 tries 1, 99)*

Versions
*Python, NumPy, pandas, scikit-learn*

**Anyone with this record should get your numbers. After cleaning in Class 1, 88 rows remain: 58, 15, 15.**

The notebook prints the record for you.

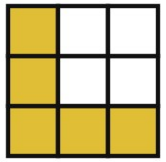
Paste it under every result you report.

# Trust comes from four habits



**Held-out data**

Score on rows the model never saw



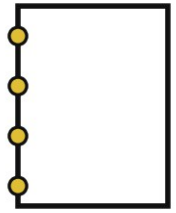
**Cross-validation**

Report mean and standard deviation



**No leakage**

Fit every preprocessing step on the training set



**Reproducibility**

Record seed, file, versions

Next class: features and choosing a model. These habits are the rulebook.

# Read the workflow before you write it

```
pipeline = make_pipeline(imputer, tree_model)
group_kfold = GroupKFold(n_splits=4)
cv_scores = cross_val_score(
    pipeline, X_train, y_train,
    groups=groups, cv=group_kfold,
    scoring="neg_mean_absolute_error")
print(-cv_scores.mean(), cv_scores.std())
```

## In plain words

make_pipeline chains the imputer and the tree into one model (a Pipeline).

GroupKFold(n_splits=4) makes 4 folds. Each fold holds out one equipment group as the validation fold.

cross_val_score runs the whole pipeline on every fold and returns one score per fold.

The scoring name gives the MAE with a minus sign, so we print -cv_scores.mean().

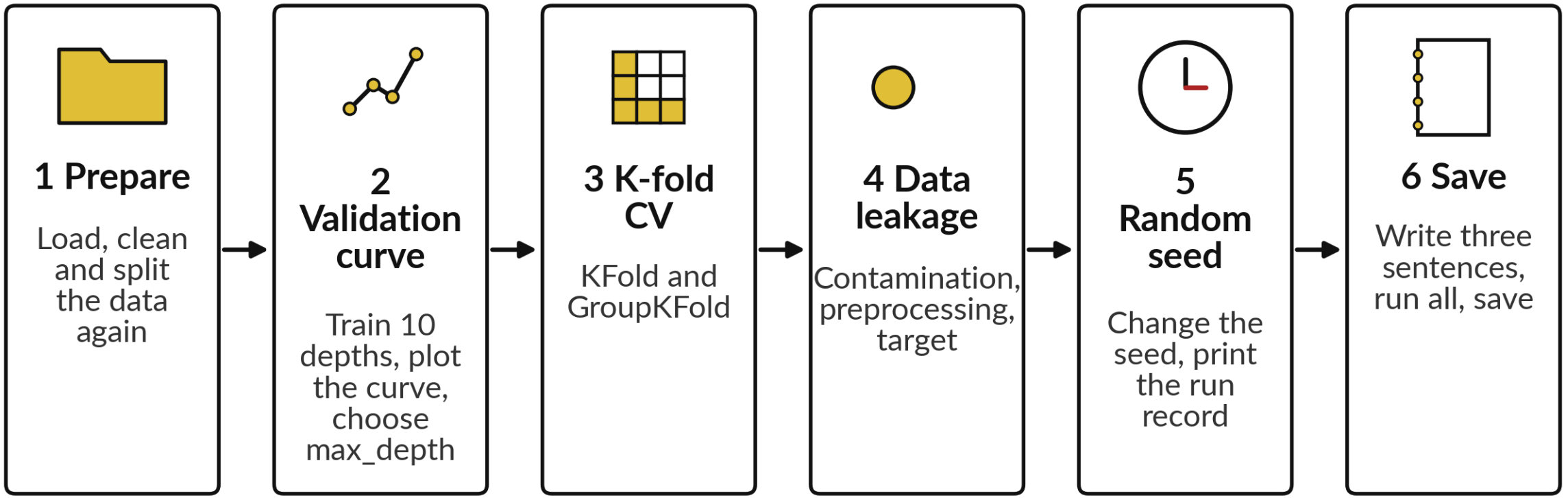
cv_scores.std() is the standard deviation of the scores.



# Part 2: the lab

Close the slides. Open the notebook. Same steps, your own data.

# The lab has six steps, and the first five end with a check




In each step, predict first, run second, then read what the output tells you.

# Check your numbers against these before you move on

| | What you print | You should see |
|---|---|---|
|  | **Rows: training, validation, test** | 58, 15, 15 |
|  | **Validation MAE at max_depth 1 and at 10** | 1.56 and 1.18 kWh/t |
|  | **Chosen max_depth** | 5 |
|  | **K-fold, k = 5: mean and standard deviation** | 1.16 and 0.24 |
|  | **GroupKFold, k = 4: mean and standard deviation** | 1.09 and 0.15 |
|  | **max_depth 10: honest and leaky validation MAE** | 1.18 and 0.01 |
|  | **max_depth 10 tree, random_state 1 and 99** | about 1.2 and about 1.3 |

MAE in kWh/t. If your number differs, stop and ask. Find the step that went wrong. The two seed numbers are about values: a difference in the second decimal is fine.

# Four problems you will meet, what they mean and what to do

| What you see | What it means | What to do |
|---|---|---|
| **NameError: X_train** | Step 1 was not run | Run all cells from the top, in order |
| **An error that mentions groups** | GroupKFold needs the group labels (a groups argument) | Pass groups=groups |
| **All scores are negative** | scikit-learn gives errors a minus sign (neg_ scoring: bigger is better) | Put a minus in front: -scores |
| **Numbers differ a little from the slide** | The random seed (random_state) is missing or different | Use random_state=17 in every tree, except in Step 5 where the seeds 1 and 99 are tested on purpose |

# How this class counts towards your coursework



Nothing is submitted today.



Block 2 is Classes 3 and 4. Its assignment has three parts: A9 (validation and trust, from today's lab), A10 (features) and A11 (model recommendation), both from Class 4.



The Block 2 assignment closes on Friday 4 December, 08:00 (Kampala time).



The Block 2 quiz opens in the last 20 minutes of Class 4, stays open for 24 hours, and covers Classes 3 and 4.

# Save the notebook and keep the checklist

## What you keep from today

Your saved lab notebook, the chosen max_depth, and the five-question checklist for trusting a score.

**Our question today**

*How do we know a model that predicts crusher energy demand will still work on a crusher it has never seen?*

**Next class**

Class 4: features and choosing a model, plus the project brief.

**Take away**

Notebook
*Restart, run all, save*

max_depth
*Chosen on the validation set*

Checklist
*Five questions*

Next class
*Features and models*